

Breaking the Mobile Bandwidth Wall for LLMs via Semantic Block Decomposition

Anonymous Author(s)

Abstract

Large Language Model (LLM) inference on mobile platforms is governed by the energy and latency costs of data movement across the memory hierarchy. While server-class accelerators utilize high-bandwidth memory (HBM), mobile SoCs are restricted by the physical throughput limits of LPDDR interfaces and small on-chip caches. In these environments, the auto-regressive decoding phase is fundamentally memory-bound, as the low operational intensity causes processing elements to remain stalled while waiting for weight data from off-chip DRAM. Conventional compression techniques often fail to provide end-to-end speedups because they overlook the overhead of irregular memory access and additional synchronization traffic.

We present **MONET**, a hardware-software co-designed system that addresses the mobile bandwidth wall through tiled block decomposition. MONET reformulates dense matrix-vector multiplications into a structured execution of basis blocks and coefficient tensors, specifically sized to align with the physical granularity of mobile memory controllers. The key insight is that the bandwidth bottleneck can be mitigated by decoupling the static model representation from the dynamic execution dataflow. We design an IO-aware execution engine that orchestrates the calculation order to maximize on-chip SRAM residency and minimizes DRAM access frequency. Furthermore, MONET exploits the inherent sparsity within the decomposed factors to eliminate redundant off-chip transfers during the reconstruction phase. This co-optimization transforms the decoding process into a high-throughput pipeline by masking DRAM latency through asynchronous weight staging. Experimental results on commodity mobile SoCs demonstrate that MONET achieves $2.42\times$ and $2.52\times$ speedups in the prefill and decoding phases, respectively, elevating hardware utilization from 21% to 52% by alleviating the memory-bandwidth bottleneck.

Keywords: LLM, Mobile Device, Block Decomposition

1 Introduction

The deployment of Large Language Models (LLMs) on mobile platforms is fundamentally constrained by the energy and latency costs of data movement. While server-class accelerators rely on high-bandwidth memory (HBM) to sustain massive throughput, mobile SoCs are restricted by the physical limits of LPDDR memory systems and small on-chip

caches. In these resource-constrained environments, the performance of LLM inference is not a function of peak arithmetic throughput but is instead dictated by the efficiency of the memory hierarchy.

The primary bottleneck, which we term the Mobile Bandwidth Wall, manifests during the auto-regressive decoding phase. Because each step of inference requires loading the entire model weight set to process a single token, the operational intensity is exceptionally low. This mismatch between compute capacity and memory bandwidth ensures that the processing elements remain stalled while waiting for weight data from off-chip DRAM. Consequently, conventional optimization targets, such as reducing the number of FLOPs, provide negligible end-to-end speedups if the total volume of data transferred remains unchanged.

Existing model compression techniques often fail to address this bandwidth crisis effectively. Quantization reduces the bit-width of weights but introduces significant accuracy degradation at ultra-low precisions. Weight sparsity, although reducing the theoretical data volume, leads to irregular memory access patterns that are incompatible with the SIMD hardware of mobile NPUs. Furthermore, standard tensor decomposition methods like Singular Value Decomposition (SVD) often introduce additional intermediate activation tensors and synchronization overhead, which can inadvertently increase the total memory traffic. These methods treat the model as a collection of matrices rather than a structured execution pipeline, overlooking the opportunity to co-design the decomposition strategy with the hardware-level data flow.

To address these challenges, we propose **MONET**, a hardware-software co-designed system that restructures LLM inference into a bandwidth-efficient paradigm via tiled block decomposition. MONET departs from monolithic matrix-vector operations by representing weight tensors as a collection of basis blocks and coefficient tensors which are structured to align with the physical granularity of mobile memory controllers.

The key insight behind our MONET is that the mobile bandwidth bottleneck can be mitigated by decoupling the static model representation from the dynamic execution dataflow. We design an IO-aware execution engine that orchestrates the computation order to minimize expensive off-chip memory transactions. Drawing inspiration from the tiling principles of FlashAttention, MONET optimizes the dataflow to maximize on-chip SRAM residency and ensures that the processing elements remain utilized while

minimizing DRAM access frequency. Furthermore, MONET exploits the inherent sparsity within the decomposed factors by selectively discarding redundant data movement associated with near-zero coefficients and basis elements. The co-design of block-structured weight representations and IO-aware execution schedules transforms the decoding process into a high-throughput pipeline. By masking DRAM latency through asynchronous weight staging and eliminating redundant transfers via factor sparsity, MONET significantly improves hardware utilization on bandwidth-limited mobile SoCs from 21% to 52%. This shift fundamentally redefines the inference bottleneck, moving the system constraints from the memory wall to the peak compute capacity of the processing units.

In this paper, we make the following contributions:

- We identify the critical architectural bottlenecks of mobile memory subsystems that cause conventional compression schemes to fail in delivering end-to-end speedups. To address this, we introduce **MONET**, a framework that employs semantic block decomposition to restructure dense weight matrices into hardware-aligned tiled forms, ensuring high mathematical fidelity while optimizing for memory-access granularity.
- We design an IO-aware dataflow that restructures the execution of block-decomposed models to minimize expensive off-chip memory transactions. By adopting a tiling strategy inspired by FlashAttention, our system orchestrates the calculation order to maximize on-chip SRAM residency and ensures that the processing elements remain fed with data while minimizing DRAM access frequency.
- We further optimize the system by leveraging the inherent sparsity within the decomposed factors, specifically the basis blocks and their associated coefficient tensors. This approach allows MONET to dynamically prune redundant data transfers and eliminate unnecessary arithmetic cycles, significantly reducing the effective bandwidth pressure and computational overhead during inference.
- We implement MONET on commodity mobile SoCs across diverse LLMs. Experimental results show that aligning the decomposition strategy with the hardware-level dataflow yields $2.42\times$ and $2.52\times$ speedups in the prefill and decoding phases, respectively. These performance improvements are achieved with strict parity in model accuracy relative to the baselines.

2 Background

2.1 Mobile LLM Inference

Recent advances in lightweight LLMs have enabled their deployment on mobile devices, driven by privacy concerns and the demand for on-device intelligence. Compared to

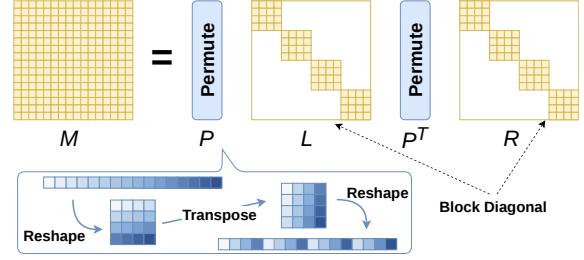


Figure 1. An example of Monarch parameterization. Monarch matrices are parametrized as products of two block-diagonal matrices up to permutation, allowing efficient multiplication algorithm that leverages batch matrix multiply.

cloud-based inference, mobile deployment avoids data transmission and supports latency-sensitive applications such as UI automation and personalized assistance [23]. However, achieving efficient inference on mobile platforms remains challenging due to limited compute capability, constrained parallelism, and restricted memory bandwidth, which together prevent effective scaling of computation and make data movement the dominant performance bottleneck.

To address these challenges, various optimization techniques have been proposed. Compression methods such as quantization aim to reduce memory footprint and model size [11, 13, 19]. Token pruning techniques reduce computation by eliminating less important tokens during inference [2, 20]. Speculative decoding accelerates token generation by leveraging smaller auxiliary models [10].

While effective in reducing computation or model size, these approaches do not fundamentally address the memory bandwidth bottleneck or the inefficiency of data movement on mobile platforms. This highlights the need for system-level solutions that optimize execution patterns and data access to achieve efficient on-device LLM inference.

2.2 Tensor Decomposition

Tensor decomposition is a widely used technique for reducing the computational and storage cost of neural networks by approximating dense weight matrices with a set of smaller factors.

A representative approach is low-rank decomposition SVD, which factorizes a rank- k weight matrix $W \in R^{m \times n}$ into two smaller matrices, reducing the computational complexity from $O(mn)$ to $O(k(m+n))$. Such low-rank methods have been widely adopted for model compression and acceleration. Extensions including Tucker [14, 25] and CP decomposition [16, 29] further generalize this idea to higher-order tensors.

Another line of work focuses on structured sparse matrices, which aim to achieve both hardware efficiency and parameter reduction on modern accelerators [7, 27]. Early

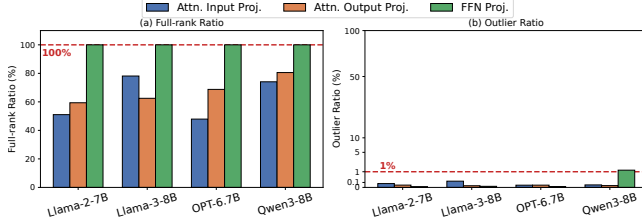


Figure 2. Rank and outlier percentage of weight across representative LLMs. (a) shows FFN layers keep full-rank, and attention projections exhibit partial rank deficiency; (b) shows weight are highly sparse with few outliers.

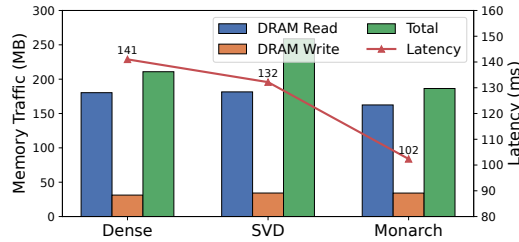


Figure 3. Comparison of memory traffic and latency across dense, SVD, and Monarch decomposition, showing that SVD increases memory traffic while Monarch reduces it and achieves lower latency. The mobile GPU is ARM Mali-G925-Immortalis.

approaches such as Butterfly matrices [17] provide high expressiveness but are often inefficient in practice due to irregular structures. To improve hardware efficiency, variants such as Pixelated Butterfly [5] have been proposed.

A prominent example in this direction is the Monarch matrix [6]. As shown in Figure 1, it represents a matrix as a product of structured components:

$$M = PLP^T R,$$

where P is a fixed permutation matrix, and L and R are block-diagonal matrices. This formulation enables efficient computation using hardware-friendly block matrix multiplications [8, 9, 18], while maintaining strong expressiveness. Moreover, Monarch supports efficient initialization from pre-trained dense weights via analytical projection.

3 Observation and Motivation

3.1 Observation

To identify the architectural bottlenecks of on-device LLM execution, we analyze both the structural properties of weight matrices and the corresponding hardware performance on mobile SoCs.

Observation 1. LLM weight matrices lack consistent low-rank structure and exhibit outlier-dominated distributions.

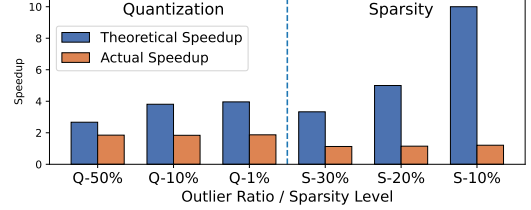


Figure 4. Theoretical vs. actual speedup on mobile devices.

Tensor decomposition is often proposed as a method to reduce the footprint of dense GEMM operations. Singular Value Decomposition (SVD) is a common choice for approximating matrices by truncating lower singular values. However, its utility in mobile LLM inference is constrained by two factors.

First, the mathematical assumption of low-rank structure is inconsistent across LLM layers. As illustrated in Figure 2(a), a significant portion of weight matrices remain full-rank. This is particularly evident in Feed-Forward Network (FFN) layers which resist rank reduction without substantial loss in fidelity. Consequently, SVD yields uneven approximation quality and forces a trade-off between model accuracy and compression ratio.

Second, low-rank factorization introduces additional intermediate activations which increase the total volume of memory traffic. As shown in Figure 3, executing two factorized matrices instead of one dense matrix incurs higher metadata overhead and materialization costs. On bandwidth-constrained mobile SoCs, the latency added by loading these additional factors often offsets the reduction in arithmetic operations.

Analysis of the weight value distribution reveals a highly skewed pattern. As shown in Figure 2(b), a small fraction of high-magnitude elements (outliers) dominates the weight distribution. These outliers are critical for model accuracy but are sparsely distributed. This non-uniform structure suggests that a block-based representation, rather than a global low-rank approximation, is required to capture local importance while maintaining hardware-friendly access patterns.

Observation 2. Outlier-aware compression techniques exhibit a fundamental misalignment with mobile SIMD hardware.

Building on the identified structural properties, existing research employs outlier-aware quantization or unstructured sparsity to reduce memory footprint. These methods prioritize the preservation of high-magnitude elements while compressing or skipping the remaining weights.

However, these techniques fail to provide proportional end-to-end speedups on mobile platforms. Figure 4 demonstrates a widening gap between theoretical FLOP reduction and measured performance gains as compression becomes more aggressive. This mismatch arises because irregular sparsity and mixed-precision quantization lead to fragmented

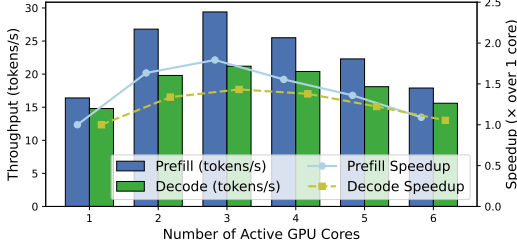


Figure 5. Throughput of mobile LLM inference under different numbers of active GPU cores. Performance saturates and degrades as more cores are enabled, indicating that inference is limited by memory bandwidth rather than compute capacity. We use Qwen-2-1.5B-Q8 as model and evaluate on Wikitext-2.

memory accesses and divergent execution paths. Mobile NPUs and GPUs rely on rigid SIMD (Single Instruction, Multiple Data) execution models. When compression schemes introduce fine-grained irregularities, the hardware suffers from low utilization due to masked lanes and uncoalesced DRAM transactions.

Observation 3. Mobile LLM inference is fundamentally limited by memory bandwidth rather than compute throughput.

We evaluate the scalability of LLM inference by varying the available compute resources on the mobile SoC. On server-class hardware, performance typically scales with increased parallelism. In contrast, mobile execution behavior reveals a different constraint.

As shown in Figure 5, increasing the active GPU core count beyond a certain threshold yields diminishing returns and eventually leads to throughput saturation. This plateau indicates that the processing elements are frequently stalled, awaiting data from the memory subsystem. Because mobile inference typically operates with a batch size of one, the operational intensity of the decoding phase is exceptionally low. Each weight is loaded from DRAM to perform a single multiply-accumulate operation, making the execution time a function of memory bandwidth rather than peak arithmetic capacity.

Therefore, optimizing for arithmetic complexity is insufficient. Performance gains in the mobile environment must come from a reduction in total DRAM transactions and the maximization of on-chip data residency.

3.2 Motivation

The deployment of LLMs at the edge requires a paradigm shift in how model weights are represented and executed. Current optimizations are often siloed, either focusing on mathematical approximation (SVD) or hardware-agnostic compression (Sparsity). To break the mobile bandwidth wall,

an ideal solution must align the non-uniform weight structure with the physical constraints of the mobile memory hierarchy.

We are motivated to explore block decomposition as a mechanism to reorganize matrix computation into fine-grained, hardware-aligned units. Unlike global low-rank methods, block-level decomposition can adapt to the local outlier density of the weights. However, existing structured decompositions often require expensive permutation or rearrangement steps which incur additional memory overhead.

Insight. The key insight of this work is that the bandwidth bottleneck can be mitigated by decoupling the static model representation from the dynamic execution dataflow. This motivates the design of MONET, a system that reformulates LLM inference into a block-centric streaming paradigm. By aligning weight blocks with the granularity of memory controllers and orchestrating an IO-aware execution schedule, we enable the system to hide DRAM latency and maximize hardware utilization.

4 MONET Framework

4.1 Overview

MONET reorganizes LLM inference as a block-centric execution pipeline that manages computation and data movement under hardware constraints. Rather than executing dense matrix operations, MONET decomposes them into block-level units and schedules their execution to enable fine-grained data access and efficient memory usage.

Specifically, MONET consists of three tightly coupled components:

- **Architecture-Aware Block Decomposition.** Transforms dense weight matrices into block-level computation units that align with hardware execution granularity.
- **Bandwidth-Aware Streaming Execution.** Executes blocks in a streaming manner, eliminating intermediate materialization and avoiding redundant data movement.
- **Cost-Aware Offline Block Reduction.** Reduces execution cost by selectively removing low-impact blocks based on offline profiling under accuracy constraints.

Together, these components form a unified execution pipeline that reduces memory traffic and improves end-to-end efficiency for bandwidth-constrained mobile inference.

4.2 Architecture-Aware Block Decomposition

MONET restructures dense matrix computations into block-level units, where the block size plays a critical role in determining execution efficiency on mobile platforms. In particular, block size directly affects both compute utilization and memory behavior.

Larger blocks improve compute efficiency by increasing arithmetic intensity and better matching the granularity

of hardware compute units. However, they also increase memory pressure, as larger intermediate data requires more on-chip buffer capacity and leads to higher off-chip memory traffic. In contrast, smaller blocks enable fine-grained data access and better memory locality, allowing more efficient streaming execution under bandwidth constraints, but may underutilize compute resources.

This introduces a fundamental trade-off between compute utilization and memory efficiency. To balance this trade-off, MONET adopts an architecture-aware block decomposition strategy that selects block sizes based on hardware characteristics. Specifically, we profile different block sizes on target hardware and measure both execution latency and memory behavior. Based on this profiling, we select the configuration that achieves the best balance between compute efficiency and memory traffic.

Furthermore, we allow block sizes to vary across layers to account for differences in workload characteristics and sensitivity. This enables MONET to adapt decomposition granularity to both model structure and hardware constraints, providing an efficient foundation for subsequent execution optimizations.

4.3 Bandwidth-Aware Streaming Execution

While block decomposition exposes structured computation, naively executing it still incurs significant memory overhead. The origin computation is:

$$y = Mx = PLP^T Rx \quad (1)$$

And a straightforward implementation materializes intermediate tensors at each stage:

$$x \rightarrow R \rightarrow u \rightarrow P^T \rightarrow v \rightarrow L \rightarrow w \rightarrow P \rightarrow y \quad (2)$$

Each step requires full-tensor reads and writes, leading to excessive memory usage and I/O. In particular, permutation operations (P^T and P) introduce substantial data movement without performing useful computation, making the execution inefficient on bandwidth-constrained mobile platforms.

The key observation is that permutation operations do not involve arithmetic computation, but only reorganize data layout. Therefore, they can be eliminated by directly writing intermediate results to their final destinations.

Inspired by streaming execution paradigms such as FlashAttention, we further organize computation in a block-wise streaming manner. Figure 6 illustrates the key difference between naive execution and our streaming execution design. Specifically, MONET treats each block as the basic unit and performs computation through streaming execution to minimize memory movement.

Let $N = b_1 \times b_2$. We partition the input as:

$$x = [x_1, x_2, \dots, x_{b_1}], \quad x_i \in \mathbb{R}^{b_2} \quad (3)$$

The first stage computes:

$$u_i = R_i x_i \quad (4)$$

Instead of materializing u and then applying P^T , we directly scatter each element $u_i^{(j)}$ into its permuted destination, forming blocks:

$$v_j = [u_1^{(j)}, u_2^{(j)}, \dots, u_{b_1}^{(j)}]^T \quad (5)$$

This eliminates the need for explicit permutation P^T . Similarly, after computing:

$$w_j = L_j v_j, \quad (6)$$

each element $w_j^{(i)}$ is directly written to its final position in y_i , avoiding the materialization of w and eliminating permutation P .

By this way, MONET executes computation in a block-wise streaming manner. For each block of R , the corresponding input slice is loaded into on-chip memory, and the block multiplication is performed. The results are immediately scattered into the appropriate buffers for subsequent L -block computation. Once sufficient data for a given L -block is available, its computation is triggered, and the results are written directly to their final positions.

This execution strategy has three key advantages. First, it eliminates explicit permutation operations by fusing them into data placement, thereby avoiding additional memory reads and writes. Second, it avoids materializing intermediate tensors, significantly reducing off-chip memory traffic. Third, it enables a streaming execution pattern in which each block is loaded, processed, and written once, maximizing on-chip data reuse. The algorithm flow is shown in Algorithm 1.

4.4 Cost-Aware Offline Block Reduction

While MONET enables efficient block-wise streaming execution, each block still incurs non-negligible memory access and computation cost. As a result, the overall inference latency remains proportional to the number of executed blocks. This motivates a further optimization: reducing the number of executed blocks while preserving model accuracy.

To understand the impact of block-level computation, we conduct a profiling study by progressively removing blocks and measuring the corresponding perplexity and latency. We consider a representative setting with matrix size 4096 and evaluate different block sizes (32 and 64). For each configuration, we vary the number of retained blocks and measure both accuracy and execution speedup, as shown in Figure 7.

We observe that model accuracy degrades gradually as blocks are removed, rather than collapsing abruptly. This indicates that block importance is highly uneven, where a significant portion of blocks contributes marginally to the final output. This property enables effective block-level pruning, allowing us to reduce both computation and memory traffic while maintaining acceptable accuracy.

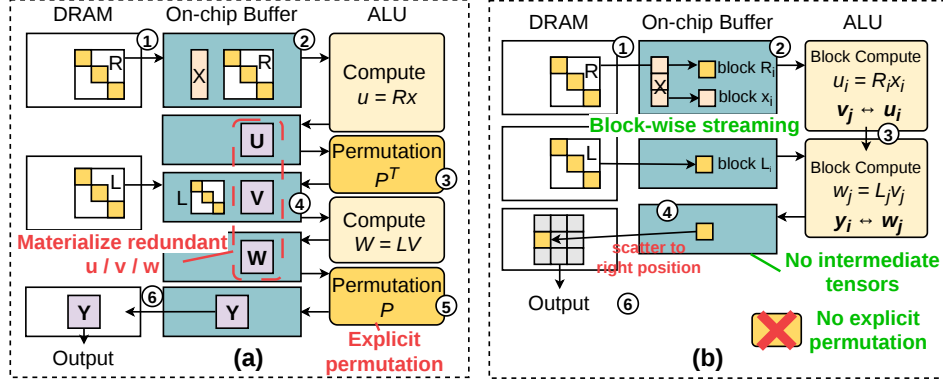


Figure 6. Bandwidth-aware streaming execution in MONET. (a) Naive execution materializes intermediate tensors (u, v, w) and performs explicit permutation operations, leading to repeated off-chip memory transfers. (b) MONET eliminates explicit permutations by directly scattering intermediate results to their destination buffers and writing outputs to final positions. This enables a block-wise streaming dataflow that minimizes memory movement and maximizes on-chip reuse.

Algorithm 1 MONET Streaming Dataflow

Require: Input x , block-diagonal factors $R = \text{blkdiag}(R_1, \dots, R_{b_1})$ and $L = \text{blkdiag}(L_1, \dots, L_{b_2})$
Ensure: Output $y = PLP^T R x$

- 1: Partition x into blocks $\{x_i\}_{i=1}^{b_1}$
- 2: Allocate intermediate block buffers $\{v_j\}_{j=1}^{b_2}$
- 3: Allocate output buffers $\{y_i\}_{i=1}^{b_1}$
- 4: **for** $i = 1$ to b_1 **do**
- 5: Load x_i and R_i
- 6: $u_i \leftarrow R_i x_i$
- 7: **for** each element $u_i^{(j)}$ in u_i **do**
- 8: Scatter $u_i^{(j)}$ into its permuted destination in buffer v_j
- 9: **end for**
- 10: **end for**
- 11: **for** $j = 1$ to b_2 **do**
- 12: Load v_j and L_j
- 13: $w_j \leftarrow L_j v_j$
- 14: **for** each element $w_j^{(i)}$ in w_j **do**
- 15: Write $w_j^{(i)}$ into its final permuted position in output block y_i
- 16: **end for**
- 17: **end for**
- 18: Concatenate output blocks $\{y_i\}$ into y
- 19: **return** y

Based on this observation, we design an offline block selection mechanism that determines a fixed execution mask prior to deployment. Specifically, we estimate block importance using a small calibration dataset.

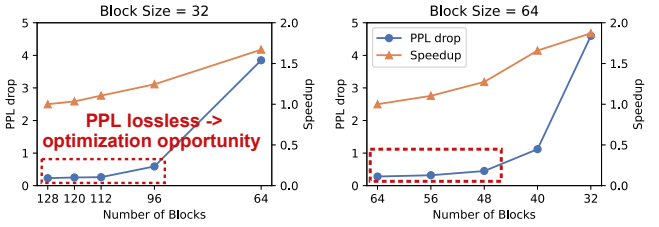


Figure 7. Impact of block pruning on accuracy and performance. We reduce the number of blocks under different block sizes and measure the corresponding perplexity and speedup. The results show that model accuracy degrades gradually as fewer blocks are retained, indicating uneven block importance.

Given a block-structured linear transformation:

$$y = \sum_{i=1}^B f_i(x),$$

where $f_i(x)$ denotes the contribution of the i -th block, we quantify block importance as the expected output contribution:

$$S_i = \mathbb{E}_{x \sim \mathcal{D}} [\|f_i(x)\|_2^2].$$

In practice, for linear blocks with $f_i(x) = W_i x_i$, we estimate the importance score as:

$$S_i \approx \frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \|W_i x_i\|_2^2,$$

which can be simply computed via a forward pass on the calibration samples.

Equivalently, this can be expressed in a compact form as:

$$S_i = \text{tr}(W_i \Sigma_i W_i^T),$$

where $\Sigma_i = \mathbb{E}[x_i x_i^T]$ is the input covariance estimated from the same dataset.

Table 1. Mobile hardware configuration.

Compon.	Device #1	Device #2
CPU	10-core ARM	8-core ARM
GPU	Mali-G925	Adreno 750
GPU Freq.	up to 1100 MHz	up to 770 MHz
Memory	16 GB	12 GB
OS	Android 15	Android 14
Framework	llama.cpp (GPU)	

This formulation captures both weight magnitude and activation statistics, providing a practical and efficient approximation of block importance without requiring exhaustive search.

We then construct a binary execution mask:

$$m_i = \begin{cases} 1, & S_i \geq \tau, \\ 0, & \text{otherwise,} \end{cases}$$

where τ controls the accuracy–efficiency trade-off. This selection is performed entirely offline, and the resulting mask remains fixed during inference.

To further align with system objectives, we incorporate block execution cost C_i (e.g., memory traffic or latency) and formulate a cost-aware selection problem:

$$\max_{\{m_i\}} \sum_i m_i S_i \quad \text{s.t.} \quad \sum_i m_i C_i \leq \mathcal{B}.$$

The resulting block mask is fixed offline and applied during inference. Since MONET operates at block granularity, skipping a block directly eliminates both weight loading and computation without introducing irregular memory access or additional overhead. As a result, this optimization effectively reduces memory traffic and improves end-to-end performance on bandwidth-constrained mobile platforms.

5 Evaluation

5.1 Experimental Setup

Hardware Platform. We conduct experiments on two smartphone devices, and summarize its hardware configuration in Table 1. By default, we use Device #1 as the main evaluation platform for its superior performance. All experiments are performed using the GPU backend of llama.cpp without NPU acceleration. All methods are evaluated under the same hardware, runtime, and quantization settings to ensure fair comparison, and reported results are averaged over multiple runs.

Models and Datasets. We evaluate MONET on representative open-source LLMs across different model scales.

Table 2. Accuracy comparison across models. MONET preserves model quality across different scales.

	Dense	SVD	NBD	SparseGPT	MONET
PPL (WikiText-2)					
LLaMA-2-7B-q6	6.12	6.48	6.31	6.75	6.47
Qwen-2.5-7B-q6	5.88	6.20	6.05	6.40	6.14
LLaMA-2-7B-q4	7.45	8.32	7.88	8.76	7.72
Qwen-2.5-7B-q4	7.10	7.95	7.52	8.20	7.38
MMLU Accuracy (%)					
LLaMA-2-7B-q6	46.8	45.1	46.0	45.6	45.6
Qwen-2.5-7B-q6	51.2	49.8	50.5	49.8	50.1
LLaMA-2-7B-q4	44.2	41.8	43.1	42.0	43.5
Qwen-2.5-7B-q4	48.5	46.0	47.2	46.3	47.6

We use mainstream models such as LLaMA-2-7B and Qwen-2.5-7B, and we adopt quantized models with 4-bit and 6-bit weight formats (Q4 and Q6) to reflect practical mobile deployment. We further include smaller models, such as TinyLlama-1.1B and Qwen-3-0.6B, to capture emerging on-device deployment trends and explore the trade-off between efficiency and accuracy under different model scales.

We evaluate both language modeling quality and downstream task performance. Perplexity is measured on WikiText-2, and reasoning ability is evaluated on MMLU.

Baselines. We compare MONET against several representative approaches: (1). Dense, we use the original model executed with the llama.cpp GPU backend without any modification as baselines. (2). SVD, We apply SVD to linear layers. The decomposition rank is tuned to achieve a similar accuracy level as MONET. (3). Naive Block Decomposition (NBD), We apply Monarch-style block decomposition by selecting block sizes that are close to the square root of the matrix dimension, aiming to maximize sparsity and reduce the number of active blocks. The resulting blocks are executed normally without streaming optimization or dataflow modification. (4) Structured Pruning, we apply one-shot structured pruning using SparseGPT, which removes weights based on Hessian-aware importance estimation. We maintain the same sparsity as MONET

Configuration Details. We implement MONET by modifying llama.cpp to support block decomposition, streaming execution, and block-level selection. All optimizations are applied at the operator level without relying on specialized hardware support. We use a small calibration dataset to determine block importance for offline block selection.

By default, we use a block size of 64 and retain 75% of blocks after selection. The calibration dataset consists of 256 samples randomly drawn from the training split of the corresponding dataset, with a sequence length of 128. All experiments are conducted with batch size 1 to reflect latency-sensitive mobile inference scenarios.

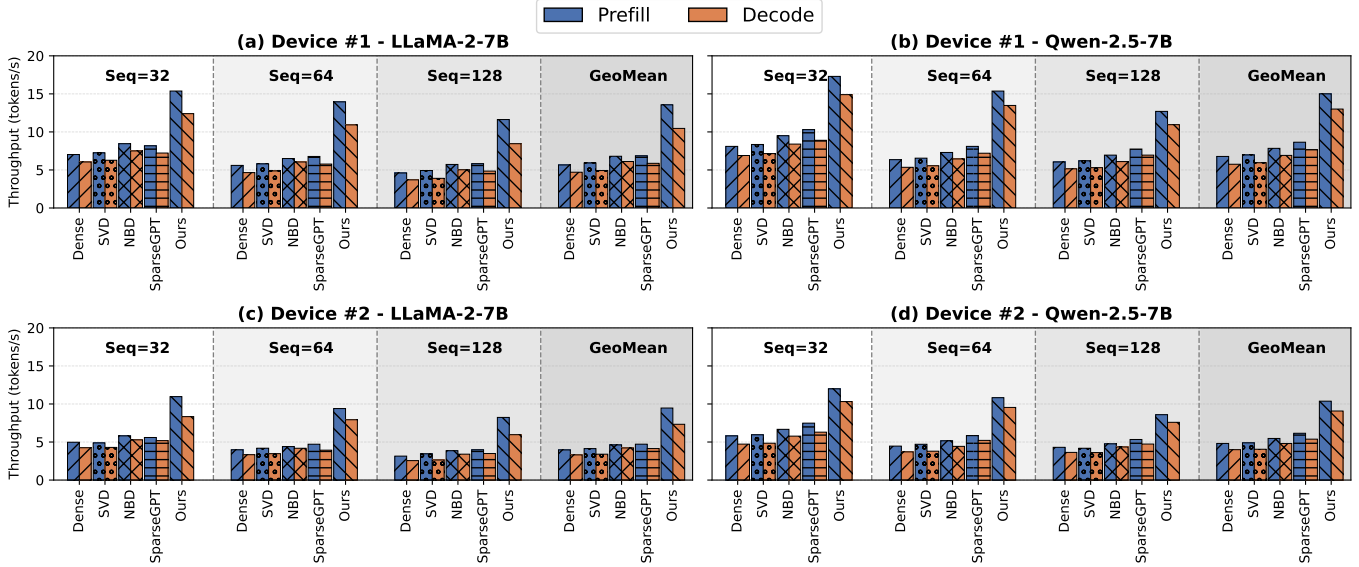


Figure 8. Prefill and decode throughput under different sequence lengths in LLaMA-2-7B and Qwen-2.5-7B.

5.2 Overall Performance

Accuracy. We first evaluate the impact of MONET on model quality across different model families and quantization settings. Table 2 reports the accuracy results for both Q6 and Q4 models. Across different models and configurations, MONET preserves model quality well and lossless. Compared with SVD and SparseGPT, which introduce noticeable degradation due to aggressive approximation or pruning, MONET achieves lower PPL and higher accuracy across all configurations. For example, on LLaMA-2-7B-q4, under the same parameter level as SparseGPT, MONET achieves better model quality with lower PPL and higher accuracy. Similar trends are observed for Qwen-2.5-7B and under lower-bit quantization (Q4), where all methods experience degradation, but MONET consistently remains closer to the dense baseline.

Importantly, MONET introduces only marginal accuracy loss compared to the dense model, while providing substantial efficiency gains. This demonstrates that MONET achieves a better accuracy–efficiency trade-off than decomposition- and pruning-based methods, making it more suitable for resource-constrained mobile deployment.

Throughput. We evaluate end-to-end inference throughput under different sequence lengths for both the prefill and decode stages. Figure 8 compares MONET and other baselines across input lengths of 32, 64, and 128 tokens. These sequence lengths are chosen to reflect practical mobile workloads, where output lengths are typically limited. In addition, longer sequences on mobile devices tend to trigger thermal throttling, leading to significant performance variance and unstable measurements.

As shown in Figure 8, MONET consistently achieves the highest throughput across both LLaMA-2-7B and Qwen-2.5-7B. For LLaMA-2-7B, MONET improves throughput by over $2\times$ across both prefill and decode stages, reaching up to $2.5\times$ speedup at longer sequence lengths. For Qwen-2.5-7B, despite its higher baseline throughput, MONET still delivers comparable gains, achieving up to $2.42\times$ speedup in prefill and $2.52\times$ in decode. These results demonstrate that MONET generalizes well across different model architectures.

Overall, MONET consistently delivers substantial performance improvements across both stages. In the prefill stage, MONET achieves approximately $2.30\times$, $2.12\times$, $2.00\times$, and $2.11\times$ speedup compared to Dense, SVD, NBD, and SparseGPT, respectively. In the decode stage, the corresponding speedups are $2.22\times$, $2.01\times$, $1.93\times$, and $2.00\times$. Summarizing across sequence lengths, MONET achieves a geometric mean speedup of approximately $2.30\times$ in prefill and $2.22\times$ in decode over the dense baseline. Notably, the improvement is more pronounced in the decode stage, where execution is more strongly constrained by memory bandwidth. By reducing redundant data movement through block-wise streaming execution, MONET effectively alleviates this bottleneck and translates algorithmic optimization into substantial end-to-end gains.

5.3 Method Efficiency Analysis

Memory Efficiency. To better understand the source of MONET’s performance gain, we analyze its impact on memory traffic. Figure 9 reports DRAM read, DRAM write, and total memory traffic for different methods.

MONET significantly reduces overall memory traffic compared with all baselines. In particular, the total traffic is reduced from 210.84 MB/token for the dense model to 118.8 MB/token, corresponding to a reduction of approximately

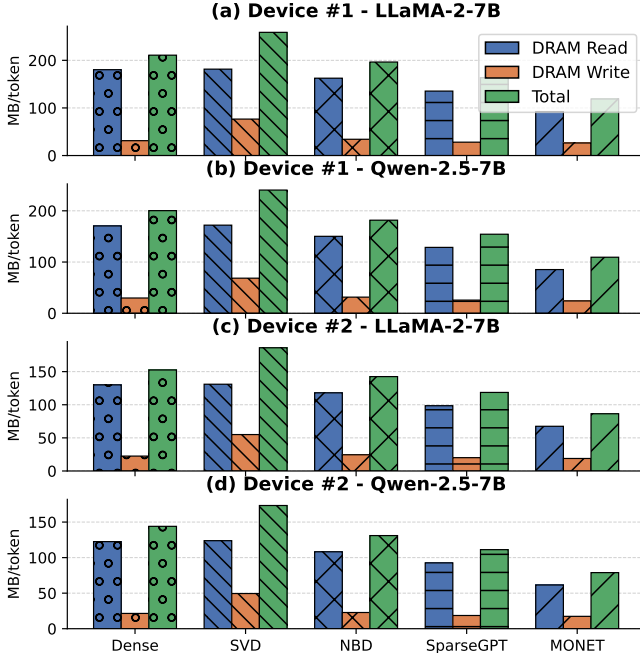


Figure 9. Memory traffic comparison results.

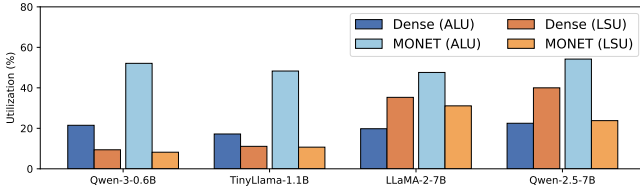


Figure 10. Utilization comparison results of mobile GPU inference on Device #1.

43.7%. In contrast, SVD increases total traffic to 258.81 MB/token due to extra intermediate representations, while NBD and SparseGPT provide only moderate reductions.

Breaking down the traffic, MONET reduces both DRAM read and DRAM write volume. The reduction in DRAM read mainly comes from avoiding redundant weight loading through block-wise streaming execution, while the reduction in DRAM write is attributed to eliminating unnecessary intermediate materialization. These results confirm that MONET directly addresses the memory bottleneck in mobile inference, rather than only reducing arithmetic complexity.

Execution Efficiency. We further analyze hardware utilization to understand how the reduction in memory traffic translates into execution efficiency. Figure 10 compares the ALU and LSU utilization of Dense and MONET across different model scales.

MONET consistently improves ALU utilization across all models. For example, on TinyLlama-1.1B, ALU utilization increases from 17.2% to 48.3%, and on Qwen-2.5-7B, it increases from 22.5% to 54.2%. This indicates that MONET

enables more continuous computation by reducing memory-induced stalls.

At the same time, MONET alleviates pressure on the LSU, especially for larger models. On LLaMA-2-7B, LSU utilization decreases from 35.3% to 31.1%, and on Qwen-2.5-7B, it is reduced more significantly from 40.0% to 23.8%. This reduction reflects fewer memory accesses and improved data locality due to block-wise streaming execution.

These results demonstrate that MONET effectively improves the compute-memory balance of mobile inference. By reducing redundant data movement, MONET not only lowers memory traffic but also allows compute units to remain active more consistently, leading to higher overall execution efficiency.

5.4 Ablation Study

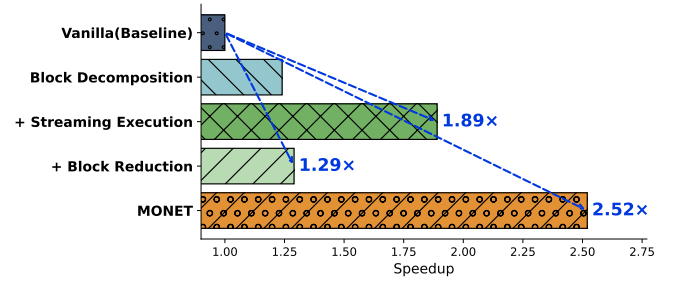


Figure 11. Impact of MONET's components on inference speedup.

Component Analysis. We evaluate the performance contributions of the three core architectural pillars in MONET: tiled block decomposition, streaming execution, and block-level reduction. As shown in Figure 11, tiled decomposition alone yields a 1.24 $\times$ speedup by improving spatial data locality and regularizing memory access. Integrating streaming execution elevates this gain to 1.89 $\times$ by successfully overlapping off-chip weight staging with arithmetic operations. Notably, the combination of block decomposition and reduction without streaming optimizations provides only a marginal 1.29 $\times$ improvement. This suggests that structural sparsity cannot be effectively exploited to reduce memory traffic without an underlying dataflow that supports fine-grained weight access. The utility of block reduction is contingent upon the streaming paradigm, where pruning weight tiles translates directly into reduced DRAM transactions and improved hardware utilization. When all components are fully integrated, MONET achieves a 2.52 $\times$ speedup over the dense baseline. These results validate that the co-optimization of model representation and execution dataflow is essential to overcome the mobile bandwidth wall and achieve near-peak throughput.

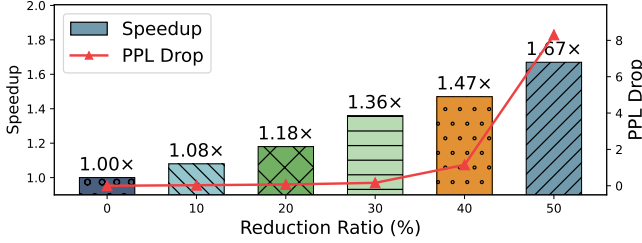


Figure 12. Impact of block size. The x-axis values denote fixed block size configurations, while "Adaptive" corresponds to layer-wise selection from the Pareto-optimal frontier.

Impact of Block Size. As shown in Figure 12, large-scale blocks maximize the utilization of wide SIMD units and improve compute efficiency by increasing the operation-to-byte ratio. However, these larger tiles expand the working set size, potentially exceeding the capacity of on-chip SRAM and triggering expensive DRAM spills. Conversely, fine-grained blocks improve cache-line utilization and reduce redundant transfers during streaming execution, yet they often suffer from hardware underutilization due to insufficient parallelism. This trade-off is aggravated by the structural variance across LLM layers, where fixed block sizes fail to achieve a global performance optimum. To resolve this, MONET employs an adaptive block strategy that selects the optimal tile size based on the physical dimensions and sparsity of each layer. By coupling this adaptive decomposition with an IO-aware dataflow, MONET balances compute throughput with memory bandwidth to achieve sustained throughput gains across the execution pipeline.

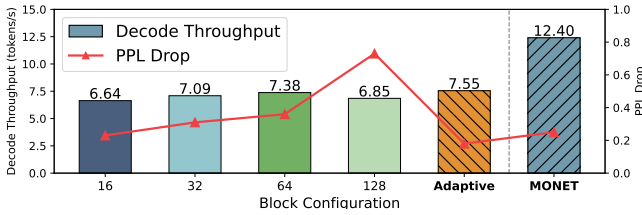


Figure 13. Impact of block reduction.

Impact of Block Reduction. We analyze the effect of block reduction on performance and model quality by varying the reduction ratio. As shown in Figure 13, increasing the reduction ratio consistently improves throughput, as fewer blocks participate in computation and data movement, thereby reducing memory traffic and execution cost. This improvement is inherently accompanied by accuracy degradation. At low reduction ratios (e.g., 10%–30%), the PPL increase remains marginal (within 0.16) while still providing noticeable speedup. As the reduction becomes more aggressive (e.g., 40%–50%), the PPL rises sharply (up to 8.30), indicating significant loss of important information.

These results highlight a fundamental trade-off between efficiency and model quality. Moderate block reduction achieves a favorable balance, whereas excessive reduction severely degrades accuracy. This observation underscores the importance of selective block reduction—removing less important blocks while preserving critical ones. MONET adopts an adaptive selection strategy to identify such blocks, enabling effective reduction while maintaining model quality.

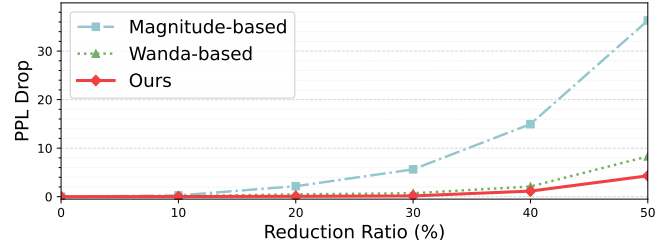


Figure 14. Effectiveness of block selection metrics.

Effectiveness of Block Selection Metrics. We evaluate the efficacy of the MONET importance metric against magnitude-based and Wanda-based heuristics across varying reduction ratios. Magnitude-based selection ranks blocks via the Frobenius norm $\|W_i\|_F$, assuming weight magnitude dictates output contribution. Wanda improves this by incorporating local activation statistics through the first-order approximation $\|W_i x_i\|$. As shown in Figure 14, magnitude-based selection incurs catastrophic perplexity (PPL) degradation as reduction ratios increase, confirming that weight magnitude is a poor proxy for block importance. While Wanda yields better results, it still suffers from significant PPL drift at high compression levels. In contrast, MONET maintains superior fidelity across all settings. At 30% reduction, MONET incurs only a 0.16 PPL increase, compared to 0.71 for Wanda and 5.63 for magnitude-based selection. These results validate that modeling the expected output contribution $\mathbb{E}[\|W_i x_i\|^2]$ provides the statistical robustness necessary to preserve model accuracy while maximizing block-level hardware reduction opportunities.

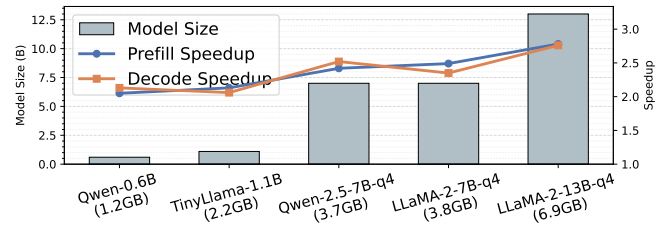


Figure 15. Scaling of MONET with model size with same accuracy loss constraint. Bars denote model parameter size, while lines show prefill and decode speedup over the origin baseline.

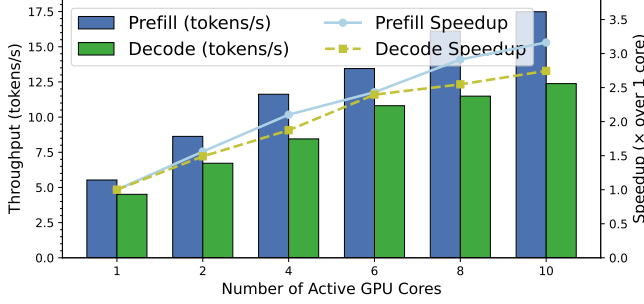


Figure 16. Scaling of MONET with GPU cores. As more cores are enabled, performance improves accordingly.

Scalability. We evaluate the scalability of MONET from both model and hardware perspectives. As shown in Figure 15, MONET achieves consistently high speedup across models from 0.6B to 13B parameters, with performance gains increasing from $2.05\times$ – $2.13\times$ on smaller models to $2.42\times$ – $2.78\times$ on larger ones. This trend is attributed to larger matrix dimensions, which not only improve the effectiveness of block decomposition, but also increase the number of blocks, providing greater flexibility for block reduction. As a result, MONET enables more aggressive reduction while maintaining accuracy, leading to higher speedup for larger models.

We further evaluate scalability with respect to GPU parallelism. As shown in Figure 16, MONET achieves consistent throughput improvement as the number of active GPU cores increases, with prefill and decode throughput improving from 5.53 to 17.49 tokens/s and from 4.51 to 12.38 tokens/s ($3.16\times$ and $2.74\times$ speedup), respectively. In contrast, the dense baseline (Figure 5) exhibits non-monotonic scaling due to memory bandwidth saturation.

This difference stems from the system bottleneck. While the dense baseline is memory-bound under increased parallelism, MONET reduces memory traffic and shifts execution toward compute-bound behavior, enabling efficient utilization of additional GPU cores. Overall, MONET scales effectively with both model size and hardware parallelism, making it particularly advantageous for large-scale models in memory-constrained environments.

6 Related Works

6.1 IO-aware and Dataflow Optimization

LLM inference performance is increasingly dictated by data movement costs across the memory hierarchy rather than raw arithmetic throughput. FlashAttention [4] and FlashDecoding++ [12] improve attention efficiency through blocked work partitioning and the reduction of intermediate activation materialization.

Complementary to kernel-level optimizations, system-level efforts such as vLLM [15] demonstrate that virtualized memory management and block-based KV-cache schemes can substantially improve throughput by reducing DRAM

fragmentation. More recently, speculative frameworks including PipeInfer [1] and Preserve [26] employ pipelined prefetching to overlap communication and memory access with computation to mitigate inference-time stalls. Unlike these works that prioritize attention mechanisms or scheduling, MONET introduces a hardware-software co-optimization that unifies weight decomposition with IO-aware streaming. This allows the system to reduce the total volume of DRAM transactions at the operator level, which is critical for bandwidth-limited mobile environments.

6.2 Mobile LLM Inference

The deployment of LLMs on resource-constrained edge devices requires navigating the strict power envelopes and restricted memory subsystems of mobile SoCs. Recent systems such as PowerInfer-2 [24] exploit neuron-cluster decomposition to support models that exceed physical memory capacity, while other efforts explore NPU offloading to accelerate the prefill stage [23]. To address hardware heterogeneity, HeteroLLM [3] and Ghidrah [21] jointly leverage GPUs and NPUs through phase-aware tensor partitioning and speculative decoding across heterogeneous processing units. These works demonstrate that mobile performance is shaped by unified memory behavior and phase-specific bottlenecks rather than raw floating-point performance. Recent surveys confirm that on-device inference remains fundamentally restricted by LPDDR bandwidth and thermal constraints [22, 28]. For example, detailed measurements of DVFS behavior on commodity mobile platforms highlight the mismatch between aggressive model scaling and stagnant memory throughput. While prior systems focus on accelerator offloading or heterogeneous workload partitioning, MONET addresses the weight-movement bottleneck within dense operators. By reformulating weights into hardware-aligned tiled forms and executing them with a streaming dataflow, MONET provides a bandwidth-efficient paradigm that minimizes redundant off-chip traffic. This approach complements existing mobile systems by directly improving the operational intensity of the underlying linear layers.

7 Conclusion

This work shows that mobile LLM inference is fundamentally bandwidth-bound rather than compute-bound. To address this, we present MONET, a system framework that optimizes LLM execution by jointly designing weight representation and execution dataflow through semantic block decomposition and streaming execution. By restructuring dense matrix operations into a bandwidth-efficient pipeline, MONET reduces memory traffic and improves hardware utilization, resulting in substantial end-to-end speedup on mobile devices. Our results suggest that optimizing data movement is key to efficient LLM inference under resource constraints, motivating co-design across model, architecture, and system.

References

- [1] Branden Butler, Sixing Yu, Arya Mazaheri, and Ali Jannesari. 2024. Pipeinfer: Accelerating llm inference using asynchronous pipelined speculation. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–19.
- [2] Han Cai, Ji Lin, Yujun Lin, Zhijian Liu, Haotian Tang, Hanrui Wang, Ligeng Zhu, and Song Han. 2022. Enable deep learning on mobile devices: Methods, systems, and applications. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 27, 3 (2022), 1–50.
- [3] Le Chen, Dahu Feng, Erhu Feng, Rong Zhao, Yingrui Wang, Yubin Xia, Haibo Chen, and Pinjie Xu. 2025. Heterollm: Accelerating large language model inference on mobile socs platform with heterogeneous ai accelerators. *arXiv e-prints* (2025), arXiv–2501.
- [4] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [5] Tri Dao, Beidi Chen, Kaizhao Liang, Jiaming Yang, Zhao Song, Atri Rudra, and Christopher Ré. 2021. Pixelated butterfly: Simple and efficient sparse training for neural network models. *arXiv preprint arXiv:2112.00029* (2021).
- [6] Tri Dao, Beidi Chen, Nimit S Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. 2022. Monarch: Expressive structured matrices for efficient and accurate training. In *International Conference on Machine Learning*. PMLR, 4690–4721.
- [7] Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. 2019. Learning fast algorithms for linear transforms using butterfly factorizations. In *International conference on machine learning*. PMLR, 1517–1527.
- [8] Hongxiang Fan, Thomas Chau, Stylianos I Venieris, Royson Lee, Alexandros Kouris, Wayne Luk, Nicholas D Lane, and Mohamed S Abdelfattah. 2022. Adaptable butterfly accelerator for attention-based NNs via hardware and algorithm co-design. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 599–615.
- [9] Daniel Y Fu, Elliot L Epstein, Eric Nguyen, Armin W Thomas, Michael Zhang, Tri Dao, Atri Rudra, and Christopher Ré. 2023. Simple hardware-efficient long convolutions for sequence modeling. In *International Conference on Machine Learning*. PMLR, 10373–10391.
- [10] Georgi Gerganov and contributors. 2023. *llama.cpp: Efficient LLM Inference in C/C++*.
- [11] Hui Guan, Shaoshan Liu, Xiaolong Ma, Wei Niu, Bin Ren, Xipeng Shen, Yanzhi Wang, and Pu Zhao. 2021. CoCoPIE: Enabling real-time AI on off-the-shelf mobile devices via compression-compilation co-design. *Commun. ACM* 64, 6 (2021), 62–68.
- [12] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhang Dong, and Yu Wang. 2023. Flashdecoding++: Faster large language model inference on gpus. *arXiv preprint arXiv:2311.01282* (2023).
- [13] Loc N Huynh, Youngki Lee, and Rajesh Krishna Balan. 2017. Deepmon: Mobile gpu-based deep learning framework for continuous vision applications. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services*. 82–95.
- [14] Yong-Deok Kim and Seungjin Choi. 2007. Nonnegative tucker decomposition. In *2007 IEEE conference on computer vision and pattern recognition*. IEEE, 1–8.
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [16] Vadim Lebedev, Yaroslav Ganin, Maksim Rakhuba, Ivan Oseledets, and Victor Lempitsky. 2014. Speeding-up convolutional neural networks using fine-tuned cp-decomposition. *arXiv preprint arXiv:1412.6553* (2014).
- [17] Yingzhou Li, Haizhao Yang, Eileen R Martin, Kenneth L Ho, and Lexing Ying. 2015. Butterfly factorization. *Multiscale Modeling & Simulation* 13, 2 (2015), 714–732.
- [18] Shiwei Liu, Chen Mu, Hao Jiang, Yunzhengmao Wang, Jinshan Zhang, Feng Lin, Keji Zhou, Qi Liu, and Chixiao Chen. 2023. Hardsea: Hybrid analog-reram clustering and digital-sram in-memory computing accelerator for dynamic sparse self-attention in transformer. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 32, 2 (2023), 269–282.
- [19] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of machine learning and systems* 5 (2023), 606–624.
- [20] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE international symposium on high-performance computer architecture (HPCA)*. IEEE, 97–110.
- [21] Jinhui Wei, Ye Huang, Yuhui Zhou, Jiazhi Jiang, and Jiangsu Du. 2025. Ghidorah: Fast LLM Inference on Edge with Speculative Decoding and Hetero-Core Parallelism. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 253–260.
- [22] Jie Xiao, Qianyi Huang, Xu Chen, and Chen Tian. 2026. Understanding large language models in your pockets: Performance study on cots mobile devices. *IEEE Transactions on Mobile Computing* (2026).
- [23] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2025. Fast on-device llm inference with npus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 445–462.
- [24] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282* (2024).
- [25] Kyeung-joo Yoon and Jong-Lark Kim. 2025. Performance Comparison of Tensor Decomposition on LLMs. *International Journal of Fuzzy Logic and Intelligent Systems* 25, 2 (2025), 136–145.
- [26] Ahmet Caner Yüzügüler, Jiawei Zhuang, and Lukas Cavigelli. 2025. Preserve: Prefetching model weights and kv-cache in distributed llm serving. *arXiv preprint arXiv:2501.08192* (2025).
- [27] Chen Zhang, Shijie Cao, Guohao Dai, Chenbo Geng, Zhuliang Yao, Wencong Xiao, Yunxin Liu, Ming Wu, Lintao Zhang, Guangyu Sun, et al. 2024. Fine-grained structured sparse computing for fpga-based ai inference. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 7 (2024), 2544–2557.
- [28] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. 2025. A review on edge large language models: Design, execution, and applications. *Comput. Surveys* 57, 8 (2025), 1–35.
- [29] Tianle Zhou, Jiakai Xu, Guanhong Liu, Jiaxiang Liu, Haonan Wang, and Eugene Wu. 2025. An approach for systematic decomposition of complex llm tasks. *arXiv preprint arXiv:2510.07772* (2025).